UADE

Exposición de experto

Temas a desarrollar

1. Repaso Greedy
2. Resolución de Ejercicios Greedy
3. Código de Huffman

Repaso Greedy

Greedy

- Un algoritmo Greedy es aquel que va construyendo la solución a partir de decisiones parciales basadas en la información disponible en cada momento. No mira hacia adelante, es decir que no ve los efectos de las decisiones a futuro y nunca reconsidera una decisión ya tomada.

- La estrategia de resolución de los problemas por Greedy se basa en identificar un conjunto de candidatos a formar parte de la solución. En cada paso se toma uno de los candidatos, el mas apropiado según un criterio definido para la solución del problema, y se evalúa si sirve o no, si sirve se agrega a la solución y si no se descarta.

- En un algoritmo Greedy se pueden identificar cuatro funciones: La función selección, La función factibilidad, La función solución y La función objetivo.

- Problema del Cambio, de la Mochila y Código de Huffman.

Síntesis Problema del Cambio

- El problema del cambio consiste en encontrar la forma de devolver un vuelto de valor **v** con monedas, utilizando la mínima cantidad de monedas. Las monedas son de denominaciones **d1; d2; …; dn,** con una cantidad infinita cada denominación.

- La función selección: siempre intentar seleccionar una moneda de mayor valor.

- La complejidad temporal es O (n), siendo n el importe para el cual devolver el cambio.

- La solución al problema del cambio dada por Greedy para el conjunto de monedas dado {$25, $10, $100, $50, $5, $1} es correcto por las denominaciones particulares de monedas, aunque para otros conjuntos de denominaciones no lo es, por lo tanto, se debe considerar que para casos distintos al indicado deberá utilizarse otra estrategia de resolución.

Síntesis Problema de la Mochila

- El problema consiste en que se tienen **n** objetos y una mochila. Para ***i = 1; 2; ..n***, el objeto ***i*** tiene un peso positivo ***Pi*** y un valor positivo ***Vi***. La mochila puede llevar un peso que no sobrepase **P**. El objetivo es llenar la mochila de tal manera que se maximice el valor de los objetos transportados, respetando la limitación de capacidad impuesta. Los objetos se pueden fraccionar.

- El criterio de selección consiste en ir utilizando el elemento con mayor relación v/p.

- La complejidad temporal es de O(n log n), siendo n la cantidad de objetos disponibles para seleccionar y completar la mochila.

- El algoritmo diseñado a partir del criterio de selección definido es óptimo en todos los casos, siempre considerando que se pueden fraccionar los elementos.

Revisión Ejercicios Prácticos

UADE

Ejercicios a Resolver

a) Estrategia de la solución aplicando Greedy e indicando cada uno de los elementos que se requieren para la resolución de esa técnica.
b) Pseudocódigo del Algoritmo de la resolución del problema, el cual debe seguir la estrategia definida en el punto a)
c) Cálculo de la Complejidad Temporal, justificando la misma en forma detallada.

1) **Planificación de tareas con plazo fijo:** Se deben procesar *n* tareas en un único procesador. Cada tarea se procesa en una unidad de tiempo y debe ser ejecutada en un plazo no superior a t_i. La tarea *i* produce una ganancia $g_i > 0$ si se procesa en un instante anterior a t_i. Una solución es factible si existe al menos una secuencia **S** de tareas que se ejecuten antes de sus respectivos plazos. Una solución óptima es aquella que maximiza la ganancia G tal que :

$$G = \sum_{s \in S} g_s$$

2) **Minimizar tiempo de espera:** Un procesador debe atender n procesos. Se conoce de antemano el tiempo que necesita cada uno de ellos. Determinar en qué orden el procesador debe atender dichos procesos para minimizar la suma del tiempo que los procesos están en el sistema.

Código de Huffman

Código de Huffman

- Los códigos de Huffman se usan para comprimir información eficientemente y logran una reducción del 20% al 90% (dependiendo de la información original).

- La información es representada como una secuencia de caracteres, donde cada carácter tiene una frecuencia conocida. Por otra parte, se supone que cada carácter es representado en binario.

- El objetivo del problema es obtener la codificación de menor tamaño.

Código de Huffman – Longitud fija y variable

- Un código de longitud fija es un código en el que todas las palabras de código tienen el mismo largo. Ejemplo: codificamos {*A, B, C,D, E, F*} con *A* → 000, *B* → 001, *C* → 010, *D* → 011, *E* → 100 y *F* →101.

- En un código de longitud variable, las palabras de código tienen diferente longitud. Ejemplo: *A* → 0, *B* → 101, *C* → 100, *D* → 111, *E* → 1101 y *F* → 1100.

- En un código sin prefijo, ninguna palabra es prefijo de otra. Ambos códigos son considerados sin prefijo.

- Supongamos que en un mensaje con 10000 símbolos la frecuencia (en miles) es A, 45; B, 13; C, 12; D, 16; E, 9; y F, 5. El código de longitud fija requiere 30000 bits; La variable la longitud uno requiere 22400 bits.

Código de Huffman - Estrategia

Criterio de Solución por *Greedy*:

- **Conjunto de candidatos**: los símbolos a utilizar para la codificación de la palabra.
- **La función selección**: se comenzarán a trabajar con los símbolos con menor frecuencia.
- **La función factibilidad**: en este caso particular no habría una limitante a validar.
- **La función solución**: haber construido la codificación (árbol de Huffman).
- **La función objetivo**: minimizar el tamaño de la codificación.

Código de Huffman – Colas de prioridad

- Una cola de prioridad es una estructura de datos que contiene un conjunto ***S*** de elementos, cada uno de ellos asociado a una prioridad.

- Hay dos tipos de colas de prioridad: máxima y mínimo. Las respectivas implementaciones son simétricas.

- Las operaciones asociadas con las colas de menor prioridad (las cuales utilizaremos) son:

  - ✓ *Insertar (S, x):* inserta el elemento x en la cola S. La complejidad de esta operación es *Θ (log n).*

  - ✓ *Extraer-Min(S)*: devuelve el elemento x con la clave más baja y lo elimina de la cola *S*. La complejidad de esta operación es *Θ (log n).*

Código de Huffman – Estrategia con árboles

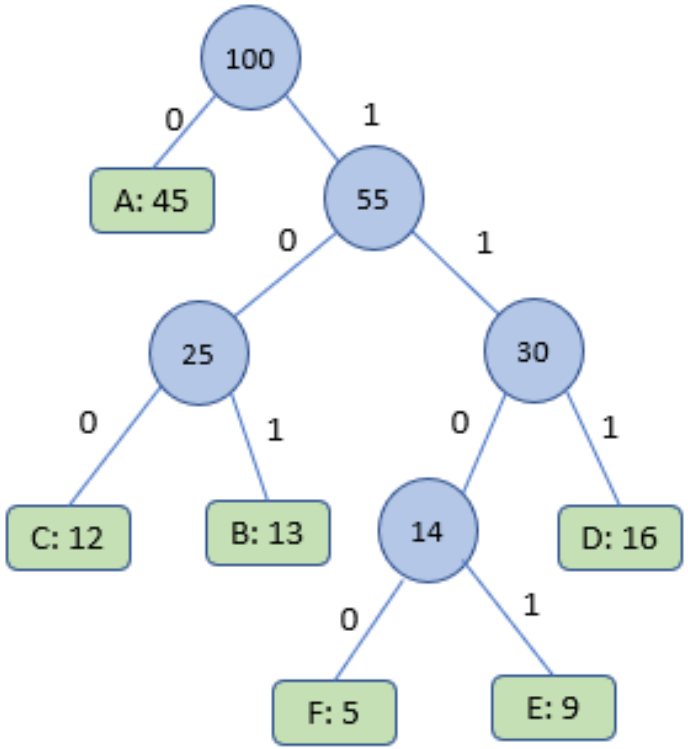
- Para llevar a cabo la estrategia descripta anteriormente la idea es construir un árbol binario comenzando por las hojas, de los símbolos a codificar.

- El árbol está construido de abajo hacia arriba. Para cada símbolo, la ruta desde la raíz proporciona la codificación.

- Cada nodo interno del árbol está asociado con una clave de la cola de prioridad.

Código de Huffman – Ejemplo

- Siguiendo el ejemplo anterior en donde en un mensaje con 10000 símbolos la frecuencia (en miles) es A, 45; B, 13; C, 12; D, 16; E, 9; y F, 5.

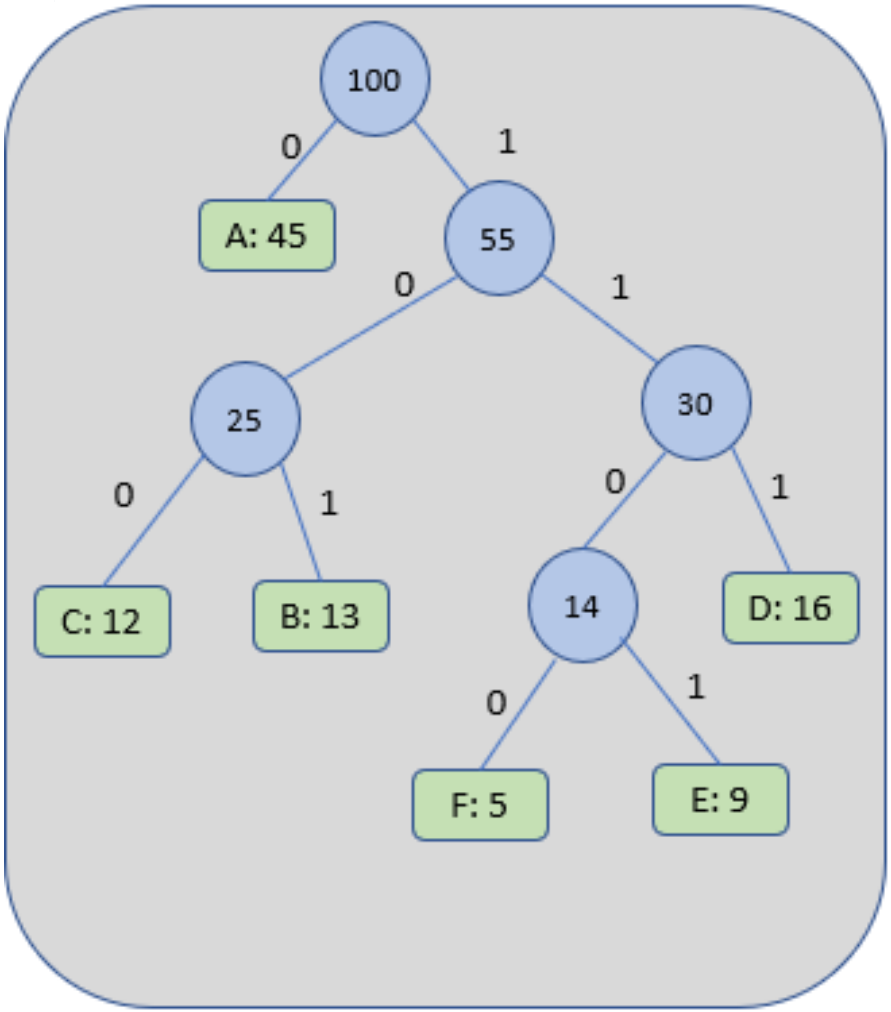


A → 0, *B* → 101, *C* → 100, *D* → 111, *E* → 1101, *F* → 1100

UADE

Código de Huffman – Ejemplo

¿Cómo se construye el árbol de Huffman? A continuación, se muestra:



<F:5, E:9, C:12, B:13, D:16, A:45>

<C:12, B:13, X1:14, D:16, A:45>

<X1:14, D:16,X2:25, A:45>

<*X*2:25, *X*3:30, A:45>

<A:45, *X*4:55>

<*X*5:100>

UADE

Código de Huffman - Algoritmo

ALGORITMO Huffman

Entrada: Conjunto C de símbolos

Salida: Z nodo raíz del árbol de codificación

```
n ← longitud (C)
Q ← InicializarColaPrioridad()
AgregarElementosACola(Q, C)
i ← 1
mientras ( i < n)
        x ← Extraer_Min(Q)
        y ← Exraer_Min(Q)
        z ← crearNodo()
        AgregarNodoIzq(z, x)
        AgregarNodoDer(z, y)
        z.frecuencia ← x.frecuencia + y.frecuencia
        Insertar(Q, z)
        i ← i + 1;
 fin mientras
devolver z;
fin Algoritmo Huffman
```

Código de Huffman – Complejidad temporal

ALGORITMO Huffman

Entrada: Conjunto C de símbolos

Salida: Z nodo raíz del árbol de codificación

```
n ← longitud (C)
Q ← InicializarColaPrioridad()
AgregarElementosACola(Q, C)                    Θ(n log n)
i ← 1
mientras ( i < n)                              Θ(n)
        x ← Extraer_Min(Q)                     Θ(log n)
        y ← Exraer_Min(Q)                      Θ(log n)
        z ← crearNodo()
        AgregarNodoIzq(z, x)
        AgregarNodoDer(z, y)
        z.frecuencia ← x.frecuencia + y.frecuencia
        Insertar(Q, z)                         Θ(log n)
        i ← i + 1;
 fin mientras
devolver z;
fin Algoritmo Huffman
```

Complejidad del Algoritmo: *Θ(n log n) +Θ(n) Θ(log n)* → ***Θ(n log n)***

¡Muchas gracias!

UADE